

Assessment 2

[Machine Learning Model Study / Project Implementation]

32513 Advanced Data Analytics Algorithms, Machine Learning – Spring 2025

Minjung Koo

14487805

ChatGpt link: <https://chatgpt.com/share/68e65c21-52f4-8010-8dc9-9287c6f3953e>

Colab link: <https://colab.research.google.com/drive/1yNoTvpkWDEimZhDOG1k6KOj5-5BmXOqL?usp=sharing>

Table of Contents

Abstract	2
Acknowledgement of AI Usage	2
1. Introduction	5
1.1 Problem.....	5
1.1.1 Motivation for choosing this topic	5
1.2 Objective	5
2. Data Preprocessing.....	6
2.2 Dataset.....	7
2.2.1 Input Features	9
2.2.2 Output Label	9
2.3 Text Vectorization (TF-IDF)	11
2.4 Evaluation Helpers and Threshold Tuning	12
2.4.1 Metric Functions	12
2.4.2. Threshold Optimisation	12
3. Model Training and Formulas	14
3.1 Logistic Regression (Baseline)	14
3.1.1 Hyperparameter Tuning / Algorithm	14
3.1.2 Loss function / Optimisation	15
3.1.3 Regularisation.....	16
3.1.4 Result	16
3.2 XGBoost (Advanced)	19
3.2.1 Hyperparameter Tuning / Algorithm	19
3.2.2 Loss function / Optimisation / Sampling.....	21
3.2.3 Regularisation.....	21
3.2.4 Result	22
4. Model Evaluation	24
4.1 Logistic Regression	24
4.1.1 Performance Summary (using threshold = 0.64)	24
4.1.2 Error Analysis (FP & FN)	24
4.1.3 Performance by Comment Length.....	25
4.2 XGBoost.....	27
4.2.1 Performance Summary (using threshold: 0.71).....	27
4.2.2 Error Analysis (FP & FN)	27
4.2.3 Performance by Comment Length.....	28

Abstract

This project aims to develop an effective system for **toxic comment classification** using both baseline and advanced models. A **Logistic Regression** model with TF-IDF features and **class weighting** was implemented as the baseline to handle class imbalance (~90% non-toxic vs 10% toxic). An **XGBoost** model was then trained to capture non-linear relationships and improve performance. Both models were tuned by adjusting the classification threshold to maximize the F1-score, which balances precision and recall. Evaluation metrics such as precision, recall, F1-score, AUC, and accuracy were compared. The results showed that XGBoost achieved a better precision–recall balance after threshold optimization, while Logistic Regression provided interpretability and fairness through class weighting.

Acknowledgement of AI Usage

I used ChatGPT to clarify key machine learning concepts such as class weighting, threshold tuning, and model evaluation metrics.

In addition, I referred to ChatGPT-generated code snippets during the model implementation process for guidance on syntax and structure.

All final code, model training, and result interpretation were conducted by me, and I critically reviewed and modified any AI-assisted outputs before inclusion.

The AI tool was used solely for educational support and not for automated code generation or report writing.

4.3 Comparison	30
4.4 Discussion	33
5. Challenges and Limitation.....	34
6. Future Work	34
7. Conclusion	35
8. References.....	36
9. Appendix	37

1. Introduction

1.1 Problem

Online platforms have become central to how people communicate, share ideas, and express opinions. However, the same openness has led to a rise in toxic comments, including hate speech, threats, and harassment. These harmful comments can negatively impact individuals and communities, making online spaces unsafe and discouraging participation. Manual moderation alone cannot handle the huge amount of content produced every second, which highlights the need for automated solutions.

1.1.1 Motivation for choosing this topic

I chose this topic because I've always been interested in how machine learning can be used to solve real-world problems. Toxic comments are everywhere on social media, and I wanted to explore how technology can make online communication safer and more positive. Working on this project also allowed me to learn more about NLP (Natural Language Processing) and compare how different models, such as Logistic Regression, XGBoost, and LSTM, handle text classification tasks. It's a meaningful area where technical knowledge can make a visible social impact.

1.2 Objective

The main objective of this project is to build and compare several machine learning models that detect and classify toxic comments. I aimed to evaluate how different algorithms perform on the same dataset, focusing on accuracy, interpretability, and computational efficiency. Specifically, I implemented Logistic Regression as a baseline and compared it with more advanced models like XGBoost and a bidirectional LSTM. By analysing their strengths and weaknesses, I wanted to identify which model provides the most balanced performance for toxicity detection.

2. Data Preprocessing

2.1 Data Collection

- Import dataset from Kaggle

```
[2]
✓ 4s !pip -q install kaggle xgboost scikit-learn pandas numpy joblib

[3]
✓ 27m from google.colab import files
files.upload() # select kaggle.json

Choose files kaggle.json
kaggle.json(application/json) - 66 bytes, last modified: 02/10/2025 - 100% done
Saving kaggle.json to kaggle.json
{'kaggle.json': b'{"username": "minjungkoo", "key": "5d2051a686f49a0f75e2eff6eea2c1e9"}'}
```

```
[4]
✓ 0s import os, shutil
os.makedirs("/root/.kaggle", exist_ok=True)
shutil.move("kaggle.json", "/root/.kaggle/kaggle.json")
os.chmod("/root/.kaggle/kaggle.json", 0o600)
```

<Figure 1: Importing the Dataset from Kaggle>

```
!kaggle datasets list -s toxic
```

ref	title	size	lastUpdated	downloadCount	voteCount
reihanenandari/youtube-toxicity-data	Youtube toxic comments	99167	2022-08-06 00:08:41.833000	4539	76
fizzbuzz/cleaned-toxic-comments	Cleaned Toxic Comments	45799147	2018-03-12 05:12:31.550000	4196	86
julian3833/jigsaw-toxic-comment-classification-challenge	jigsaw-toxic-comment-classification-challenge	55956177	2021-11-11 04:08:25.700000	13231	247
prokopjev/armories	Toxic Armories	61365	2017-02-21 00:45:26.850000	368	24
ashwinier176/toxic-tweets-dataset	Toxic Tweets Dataset	2322517	2021-05-04 05:46:27.237000	3781	46
blackmoon/russian-language-toxic-comments	Russian Language Toxic Comments	1562846	2019-05-12 10:19:56.100000	6683	92
epa/toxic-release-inventory	Toxic Release Inventory	598475499	2017-08-23 18:25:45.937000	2142	47
hanselliott/toxic-plant-classification	Toxic Plant Classification	479228680	2022-09-02 14:00:58.533000	1203	26
techsash/waste-classification-data	Waste Classification data	447914276	2019-06-16 03:24:52.733000	48264	531
alexandersemetov/toxic-russian-comments	Toxic Russian Comments	12547274	2020-11-27 04:48:24.830000	1718	28
kashnitsky/jigsaw-multilingual-toxic-test-translated	Jigsaw multilingual toxic - test & val translated	26219869	2020-03-30 16:04:03.357000	782	66
julian3833/jigsaw-multilingual-toxic-comment-classification	jigsaw-multilingual-toxic-comment-classification	1159031079	2021-11-14 04:21:43.170000	1158	35
devkhan24/toxic-comment	Toxic comment	344860489	2021-11-19 14:42:14.010000	477	16
julian3833/jigsaw-unintended-bias-in-toxicity-classification	jigsaw-unintended-bias-in-toxicity-classification	758542209	2021-11-11 05:09:51.517000	1462	32
nkitgupta/jigsaw-regression-based-data	Jigsaw Regression Based Data	2862187909	2022-02-05 20:51:56.080000	1927	115
julian3833/toxic-bert	toxic-bert	408521087	2021-11-13 03:35:07.217000	59	11
miklgr500/jigsaw-train-multilingual-coments-google-api	Jigsaw Train Multilingual Coments (Google API)	524390324	2020-04-06 13:44:59.127000	2120	95
reihanenandari/mental-health-corpus	Mental Health Corpus	4971088	2023-01-04 01:56:41.113000	6676	82
emmanuelwerrr/thyroid-disease-data	Thyroid Disease Data	148814	2022-06-09 11:27:49.700000	12539	124
fanconic/smiles-toxicity	SMILES Toxicity	1729974	2019-08-08 13:48:01.510000	909	16

```
# assumes your Kaggle API key is already set up
!mkdir -p /content/jigsaw_ml
!kaggle datasets download -d julian3833/jigsaw-multilingual-toxic-comment-classification -p /content/jigsaw_ml -q
!unzip -q -o /content/jigsaw_ml/jigsaw-multilingual-toxic-comment-classification.zip -d /content/jigsaw_ml
!ls -lh /content/jigsaw_ml | sed -n '1,120p'
```

```
Dataset URL: https://www.kaggle.com/datasets/julian3833/jigsaw-multilingual-toxic-comment-classification
License(s): CC0-1.0
total 5.0G
-rw-r--r-- 1 root root 1.1G Nov 14 2021 jigsaw-multilingual-toxic-comment-classification.zip
-rw-r--r-- 1 root root 92M Nov 14 2021 jigsaw-toxic-comment-train.csv
-rw-r--r-- 1 root root 388M Nov 14 2021 jigsaw-toxic-comment-train-processed-seqlen128.csv
-rw-r--r-- 1 root root 821M Nov 14 2021 jigsaw-unintended-bias-train.csv
-rw-r--r-- 1 root root 3.3G Nov 14 2021 jigsaw-unintended-bias-train-processed-seqlen128.csv
-rw-r--r-- 1 root root 613K Nov 14 2021 sample_submission.csv
-rw-r--r-- 1 root root 28M Nov 14 2021 test.csv
-rw-r--r-- 1 root root 488K Nov 14 2021 test_labels.csv
-rw-r--r-- 1 root root 118M Nov 14 2021 test-processed-seqlen128.csv
-rw-r--r-- 1 root root 3.1M Nov 14 2021 validation.csv
-rw-r--r-- 1 root root 15M Nov 14 2021 validation-processed-seqlen128.csv
```

<Figure 2: Dataset Selection using token from Kaggle and Download Process>

The dataset used in this project was obtained from the **Kaggle competition “Jigsaw Multilingual Toxic Comment Classification”**. The dataset repository contains multiple files, including both English and multilingual versions, as well as processed variants with different sequence lengths.

< > jigsaw-multilingual-toxic-comment-classification

Name

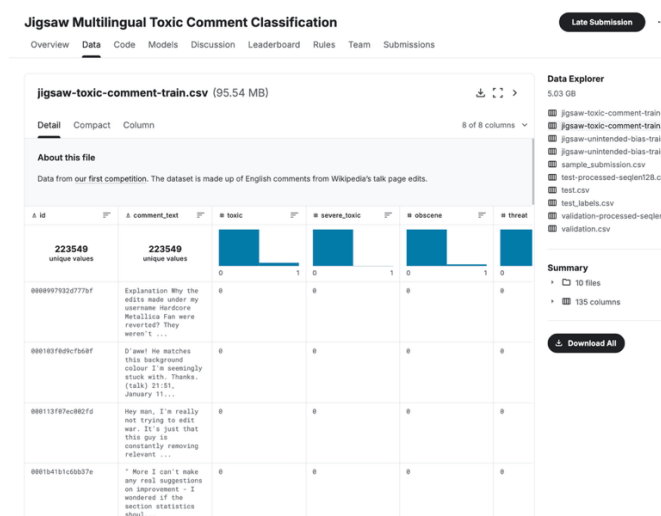
-  jigsaw-toxic-comment-train-processed-seqlen128.csv
-  jigsaw-toxic-comment-train.csv
-  jigsaw-unintended-bias-train-processed-seqlen128.csv
-  jigsaw-unintended-bias-train.csv
-  sample_submission.csv
-  test_labels.csv
-  test-processed-seqlen128.csv
-  test.csv
-  validation-processed-seqlen128.csv
-  validation.csv

<Figure 3: Files Contained in the Dataset Repository>

Among these files, this project specifically utilised **jigsaw-toxic-comment-train.csv**, which contains English comments and their binary toxicity labels (toxic = 1, non-toxic = 0). This file was chosen because it provides a sufficiently large and well-balanced training foundation for traditional machine learning models such as Logistic Regression and XGBoost, while avoiding the additional complexity introduced by multilingual or bias-related datasets (e.g., unintended-bias-train.csv).

Other files in the repository, such as `sample_submission.csv`, `test_labels.csv`, and preprocessed sequence-length versions, were excluded since the current focus was on **binary toxicity classification in English text only**.

2.2 Dataset



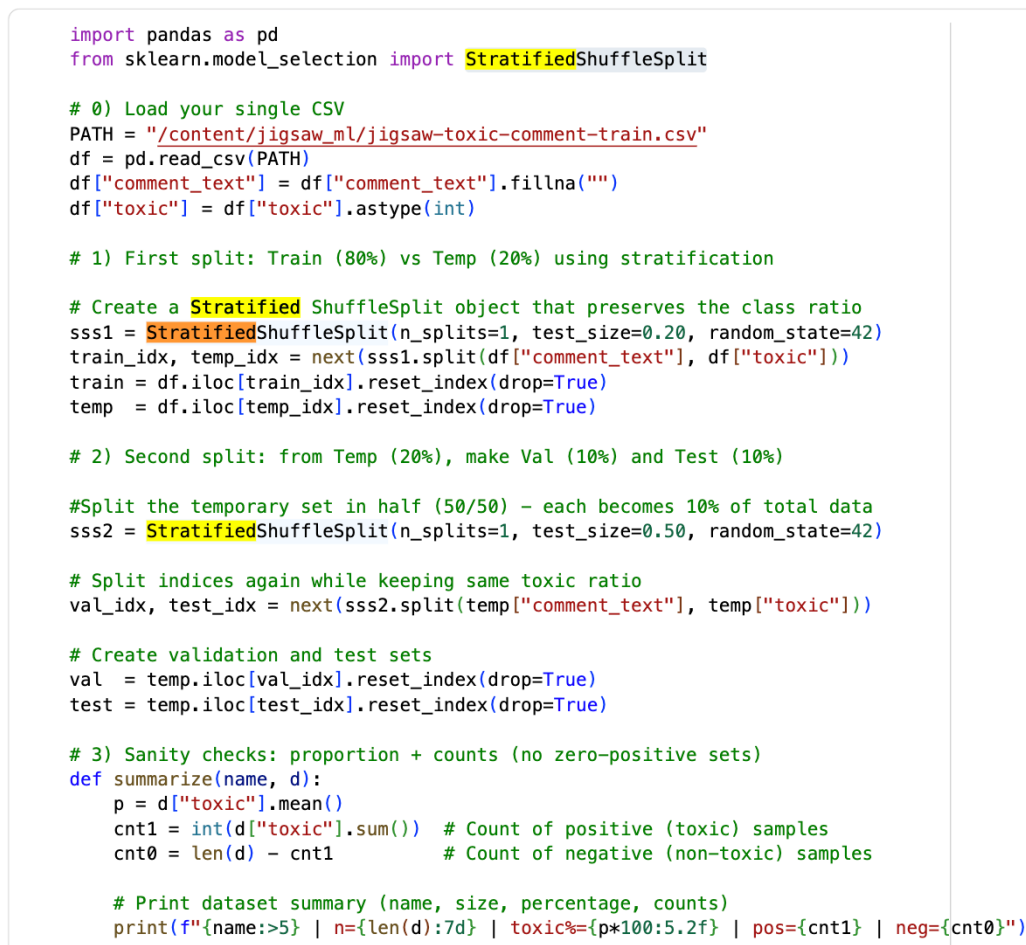
<Figure 4: Dataset Overview from Kaggle Interface>

The dataset used in this project is **Jigsaw Toxic Comment Train** from Kaggle, which contains over 22,000 labelled samples.



<Figure 5: Class Distribution of Toxic and Non-Toxic Comments>

Each comment is classified as either *toxic* (1) or *non-toxic* (0), resulting in a highly **imbalanced distribution** of approximately **10% toxic** and **90% non-toxic** samples.



<Figure 6: Stratified Sampling for Data Splitting>

To maintain the same class proportions across subsets and avoid bias during model training, the dataset was split using a **StratifiedShuffleSplit** approach. This method ensures that each split preserves the overall class ratio.

Specifically, the dataset was divided into **training (80%)**, **validation (10%)**, and **test (10%)** sets. The training set was used for model learning, the validation set for threshold tuning and hyperparameter optimisation, and the test set for final model evaluation

2.2.1 Input Features

The dataset used in this project is the *Jigsaw Multilingual Toxic Comment Classification* dataset from Kaggle, containing over 220,000 English comments collected from various online discussion platforms. Each comment is represented as raw text in the **comment_text** column. Missing values were replaced with empty strings to ensure consistency during preprocessing. The comments were later transformed into numerical feature vectors using the **TF-IDF vectorizer**, capturing both unigram and bigram features to represent word importance.

To maintain fairness across splits, the data was divided into **training (80%)**, **validation (10%)**, and **testing (10%)** sets using **stratified shuffle sampling**, which preserved the original toxic/non-toxic ratio (~90% vs 10%).

2.2.2 Output Label

The output label is a binary variable stored in the **toxic** column, where 1 represents **toxic comments** (offensive or harmful language) and 0 represents **non-toxic comments** (neutral or appropriate language).

Given that only about **9–10%** of comments are labeled as toxic, the dataset is highly imbalanced. To mitigate this imbalance, two complementary approaches were used:

```

↔ (223549, 8)
      proportion
toxic
0      0.904343
1      0.095657

dtype: float64

```

<Figure 7: Output Label and Class Balancing Techniques>

(1) **stratified sampling** during data splitting, and

(2) **class weighting** in both models, `class_weight="balanced"` for Logistic Regression and `scale_pos_weight` in XGBoost, so that toxic comments received proportionally higher importance during training.

```

↔ train | n= 178839 | toxic%= 9.57 | pos=17107 | neg=161732
   val | n=  22355 | toxic%= 9.57 | pos=2139 | neg=20216
   test | n=  22355 | toxic%= 9.56 | pos=2138 | neg=20217

```

<Figure 8: Class Distribution After Stratified Data Splitting>

2.3 Text Vectorization (TF-IDF)

To transform the raw comment texts into numerical features suitable for machine learning, the **TF-IDF (Term Frequency–Inverse Document Frequency)** technique was applied.

Vectorize text with TF-IDF (fit on train only)

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Extract target labels (0 or 1) from each split and convert to NumPy arrays
y_train = train["toxic"].astype(int).values
y_val = val["toxic"].astype(int).values
y_test = test["toxic"].astype(int).values

# Initialise a TF-IDF vectorizer to convert text into numerical features
tfidf = TfidfVectorizer(
    lowercase=True,          # Convert all text to lowercase before processing
    ngram_range=(1,2),      # Use both unigrams (single words) and bigrams (two words combinations)
    min_df=3,               # Ignore terms that appear in fewer than 3 documents
    max_features=200_000    # Limit vocabulary size to the 200,000 most frequent tokens
)

X_tr = tfidf.fit_transform(train["comment_text"].fillna(""))
X_va = tfidf.transform(val["comment_text"].fillna(""))
X_te = tfidf.transform(test["comment_text"].fillna(""))

# Check the dimensions (rows = samples, columns = features)
X_tr.shape, X_va.shape, X_te.shape
```

 ((178839, 200000), (22355, 200000), (22355, 200000))

<Figure 9: Text Vectorization Using TF-IDF>

TF-IDF captures the relative importance of words by assigning higher weights to words that appear frequently in a comment but are rare across the entire dataset.

Only the training set was used to fit the vectorizer to avoid data leakage.

The final feature matrix used **unigrams and bigrams** (`ngram_range=(1,2)`) with a minimum document frequency of 3 (`min_df=3`) and a vocabulary limit of 200,000 (`max_features=200000`).

Validation and test sets were transformed using the same fitted vectorizer.

This approach ensures that the representation remains consistent and generalizable.

2.4 Evaluation Helpers and Threshold Tuning

2.4.1 Metric Functions

```
from sklearn.metrics import (
    f1_score, precision_score, recall_score, roc_auc_score, confusion_matrix, accuracy_score
)

# Helper function to evaluate binary classification performance
def evaluate_binary(y_true, proba, threshold=0.5, label=""):

    # Convert probabilities to binary predictions based on threshold
    pred = (proba >= threshold).astype(int)

    # Print threshold and label for clarity
    print(f"[{label}] thr={threshold:.2f}")

    # Calculate and print evaluate metrics
    print(" F1 :", round(f1_score(y_true, pred), 4)) # Harmonic mean of precision and recall
    print(" P :", round(precision_score(y_true, pred), 4)) # Precision = TP / (TP + FP)
    print(" R :", round(recall_score(y_true, pred), 4)) # Recall = TP / (TP + FN)
    print(" AUC:", round(roc_auc_score(y_true, proba), 4)) # Area Under ROC Curve using probabilities
    print(" ACC:", round(accuracy_score(y_true, pred), 4)) # Accuracy = (TP + TN) / total samples

    # Compute confusion matrix to show counts of TP, TN, FP, FN
    tn, fp, fn, tp = confusion_matrix(y_true, pred).ravel()
    print(f" CM : TN={tn} FP={fp} FN={fn} TP={tp}\n") # TN, FP, FN, TP in one line
```

<Figure 10: Evaluation Helper Function for Model Metrics>

To evaluate model performance, a custom helper function was written using *scikit-learn* metrics such as **F1-score**, **precision**, **recall**, **AUC**, and **accuracy**.

The function converts predicted probabilities into binary labels based on a given threshold and prints each metric to compare model outputs clearly.

A **confusion matrix** is also generated to show counts of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN).

These metrics provide a comprehensive understanding of how well the model distinguishes between toxic and non-toxic comments, beyond simple accuracy.

2.4.2. Threshold Optimisation

```
# Helper function to find the best threshold maximising F1 score
def tune_threshold_for_f1(y_true, proba, grid=np.arange(0.10, 0.91, 0.01)):

    best_t, best_f1 = 0.5, -1.0 # Initialise best threshold and score

    for t in grid: # Iterate over threshold candidates (0.10 -> 0.9)
        f1 = f1_score(y_true, (proba >= t).astype(int))

        if f1 > best_f1: # Update when a higher F1 is found
            best_f1, best_t = f1, t

    return float(best_t), float(best_f1) # Return the best threshold and F1
```

<Figure 11: Threshold Optimisation Function for F1 Maximisation>

Because the dataset is highly imbalanced, using the default threshold of 0.5 can lead to biased results.

To address this, a threshold tuning function was implemented to find the **optimal threshold that maximizes the F1-score**.

This function iterates through candidate thresholds from 0.10 to 0.90, evaluates F1-scores for each, and returns the threshold that yields the best balance between **precision** and **recall**.

This process allows both models to fairly handle class imbalance by reducing false negatives without sacrificing too much precision.

3. Model Training and Formulas

3.1 Logistic Regression (Baseline)

3.1.1 Hyperparameter Tuning / Algorithm

To make the Logistic Regression model stable and suitable for sparse TF-IDF features, several key hyperparameters were adjusted

- **penalty='l2'** → Applies L2 regularisation to prevent overfitting by keeping model weights small
- **C=0.5** → Controls regularisation strength (smaller C = stronger penalty, helps balance bias and variance)
- **max_iter=300** → Limits training to 300 iterations, ensuring convergence during optimisation.
- **solver='liblinear'** → A solver well-suited for small or sparse datasets like TF-IDF
- **class_weight='balanced'** → Automatically adjusts for the 90:10 class imbalance between non-toxic and toxic comments

```
lr = LogisticRegression(  
    penalty="l2",  
    C=0.5,  
    max_iter=300,  
    class_weight="balanced",  
    solver="liblinear",  
)
```

<Figure 12: Logistic Regression Hyperparameter Setup>

○ Hyperparameter Summary

Parameter	Description	Value	Purpose
penalty	Regularisation type	"l2"	Reduces overfitting
C	Regularisation strength	0.5	Controls model complexity
max_iter	Max optimisation iterations	300	Ensures stable convergence

solver	Optimisation algorithm	"liblinear"	Efficient with sparse features
class_weight	Weight adjustment	"balanced"	Handles class imbalance

3.1.2 Loss function / Optimisation

Concept	Function/Method	Purpose
Loss Function	Binary Cross-Entropy	Measures difference between predicted and actual labels
Optimisation	Gradient-based (Liblinear)	Minimises loss by adjusting weights
Convergence Limit	max_iter=300	Prevents infinite loops and ensures stability

- **Loss function**
- Uses **logistic loss / binary cross-entropy**

$$\text{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

$$L = -\frac{1}{N} \sum [y \log(p) + (1 - y) \log(1 - p)]$$

<Figure 13: Binary Cross-entropy>

- Ensures output probabilities remain between **0 and 1**, suitable for binary classification
- **Optimisation method**
- **Gradient-based optimisation** (via liblinear solver)
- Iteratively adjusts weights to minimise the loss function
- Stops when convergence is achieved (within max_iter=300)
- **Purpose**
- Finds the weight values that best separate **toxic vs non-toxic** comments.
- Achieves balance between model accuracy and generalisation.

3.1.3 Regularisation

Parameter	Type	Description	Effect
penalty	L2	Adds squared weights penalty	Prevents overfitting
C	Regularisation strength	Smaller C = stronger regularisation	Balances bias–variance
Regularisation Goal	—	Keeps model simple and stable	Improves generalisation

- **Type**
 - **L2 regularisation (Ridge penalty)** → Adds the sum of squared weights to the loss function
 - Prevents any single feature from dominating the prediction
- **In this model**
 - penalty='l2' → enforces weight shrinkage
 - C=0.5 → sets moderate regularisation strength (smaller C = stronger penalty)
 - Helps balance **model bias vs variance**
- **Purpose**
 - Avoids overfitting on noisy or rare words in the TF-IDF features
 - Improves model's generalisation to unseen data

3.1.4 Result

After tuning and training the Logistic Regression model, its performance was evaluated on both validation and test sets.

The optimal **threshold (t = 0.64)** was selected based on the highest F1 score achieved during validation.

```
[LogReg (val)] thr=0.64
F1 : 0.7408
P  : 0.7118
R  : 0.7723
AUC: 0.9692
ACC: 0.9483
CM : TN=19547 FP=669 FN=487 TP=1652

[LogReg (test)] thr=0.64
F1 : 0.748
P  : 0.7244
R  : 0.7732
AUC: 0.9677
ACC: 0.9502
CM : TN=19588 FP=629 FN=485 TP=1653
```

<Figure 14: Logistic Regression Validation and Test Result (t= 0.64)>

- The model achieved **consistent performance** across validation and test sets
 - A balanced trade-off between **precision (≈ 0.72)** and **recall (≈ 0.77)** indicates strong reliability in identifying toxic comments without excessive false positives
 - The **AUC ≈ 0.97** suggests high discriminative power between toxic and non-toxic classes
 - High accuracy ($\sim 95\%$) confirms that the model generalised well beyond the training data
- **Top Features by weight**

Top +toxic features:	
stupid	16.891
fuck	16.699
fucking	15.053
shit	14.505
idiot	13.171
ass	10.583
bullshit	9.964
asshole	9.939
crap	9.836
bitch	9.810
penis	9.605
suck	9.551
dick	9.403
idiots	9.057
you	8.949
moron	8.179
sucks	8.119
gay	7.911
faggot	7.473
cunt	6.965
Top -toxic (non-toxic) features:	
thanks	-4.924
thank you	-4.252
thank	-4.146
talk	-3.903
please	-3.714
redirect	-3.262
may	-3.100
wp	-3.074
at	-3.062
if you	-3.060
cheers	-2.984
in	-2.756
to	-2.751
the	-2.663
see	-2.634
welcome	-2.565
as	-2.522
redirect talk	-2.501
appreciate	-2.480
section	-2.479

<Figure 15: Top Features by Weight in Logistic Regression>

Top Toxic Features (Positive Weights)	Top Non-Toxic Features (Negative Weights)
stupid (16.891)	thanks (-4.924)
fuck (16.699)	thank you (-4.252)
fucking (15.053)	thank (-4.146)
idiots (9.057)	please (-3.714)
asshole (9.939)	welcome (-2.565)

- Words with **high positive weights** (e.g., “*stupid*”, “*fuck*”, “*idiot*”) strongly increase toxicity prediction
 - Words with **negative weights** (e.g., “*thanks*”, “*please*”, “*welcome*”) reduce toxicity likelihood, showing polite or neutral tone
 - This confirms that the model effectively learned meaningful semantic distinctions in text tone
-
- **Overall Insight**
 - The Logistic Regression model, with balanced class weighting and L2 regularisation, provided a **robust baseline** for toxic comment classification
 - Despite dataset imbalance, it achieved **high precision-recall consistency** and interpretable word-level insights
 - These interpretable coefficients make it a solid benchmark before moving to more complex models like **XGBoost**

3.2 XGBoost (Advanced)

XGBoost was adopted as an advanced model to capture **non-linear relationships** and improve classification performance.

It uses gradient boosting to iteratively reduce log-loss while handling class imbalance efficiently

3.2.1 Hyperparameter Tuning / Algorithm

```
# Parameter settings
params = {
    "objective": "binary:logistic", # Objective function -> minimise log loss using gradient boosting
    "eval_metric": "logloss",      # Evaluation metric printed during training (lower is better)
    "max_depth": 6,                # Maximum tree depth
    "eta": 0.08,                   # Learning rate; shrink each new tree's contribution
    "subsample": 0.8,              # Row sampling (use 80% of rows per tree)
    "colsample_bytree": 0.8,       # Column Sampling (use 80% of features per tree)
    "lambda": 1.0,                # L2 regularisation on leaf weights (keep them small)
    "scale_pos_weight": spw,       # Balance positive vs negative classes
    "tree_method": "hist",         # histogram-based splits (fast & memory efficient)
    "seed": 42                     # make randomness (sampling/splits) reproducible
}
```

<Figure 16: XGBoost Model Hyperparameter Settings>

1. Hyperparameter Setting

- **objective = "binary:logistic"** → Specifies binary classification using logistic regression as the output function
- **eval_metric = "logloss"** → Uses log-loss as the evaluation metric during training; lower values indicate better predictive performance
- **max_depth = 6** → Limits the maximum depth of trees to control model complexity and prevent overfitting
- **eta = 0.08** → Defines the learning rate; a smaller value ensures gradual convergence and reduces the risk of overshooting during optimisation
- **subsample = 0.8** → Randomly samples 80 % of the training data per boosting round to introduce diversity and mitigate overfitting
- **colsample_bytree = 0.8** → Randomly selects 80 % of the features for each tree, improving model robustness
- **lambda = 1.0** → Applies L2 regularisation on leaf weights, keeping them small to avoid over-complexity
- **scale_pos_weight = spw** → Balances the ratio of positive (toxic) and negative (non-toxic) samples to address class imbalance
- **tree_method = "hist"** → Uses the histogram-based tree construction method, which accelerates computation on large datasets
- **seed = 42** → Ensures reproducibility by fixing randomisation during sampling and split operations.

```

# Training with early stopping
evallist = [(dtrain, "train"), (dval, "eval")]
bst = xgb.train(
    params=params,
    dtrain=dtrain,
    num_boost_round=300,    # max number of trees (rounds)
    evals=evallist,
    early_stopping_rounds=50, # stop if 'eval' metric doesn't improve for 50 rounds
    verbose_eval=True
)

```

<Figure 17: XGBoost Early Stopping Training Configuration>

2. Early Stopping Configuration

- **num_boost_round = 300** → Sets the maximum number of boosting iterations (trees)
- **early_stopping_rounds = 50** → Halts training if the evaluation metric (**logloss**) does not improve for 50 consecutive rounds, reducing unnecessary computation
- **evals = [(dtrain, "train"), (dval, "eval")]** → Monitors model performance on both training and validation sets during each boosting round
- **verbose_eval = True** → Enables progress logging, allowing visibility into evaluation metrics during training

○ XGBoost Model Configuration and Hyperparameter Summary

Parameter	Value	Purpose / Description
objective	"binary:logistic"	Performs binary classification using logistic loss
eval_metric	"logloss"	Evaluation metric (lower = better)
max_depth	6	Controls model complexity (limits tree growth)
eta	0.08	Learning rate; slows down updates for stability
subsample	0.8	Uses 80% of rows per tree for diversity
colsample_bytree	0.8	Uses 80% of features per tree
lambda	1.0	L2 regularisation on leaf weights
scale_pos_weight	Computed (neg/pos ratio)	Balances toxic vs non-toxic samples
tree_method	"hist"	Fast histogram-based splits

seed	42	Ensures reproducibility and consistent results
num_boost_round	300	Max boosting rounds
early_stopping_rounds	50	Stops if no improvement after 50 rounds

- **Algorithm flow**

- Builds trees sequentially; each corrects previous errors
- Uses **gradient descent** to minimise log-loss
- Monitors validation loss and stops early to avoid overfitting

3.2.2 Loss function / Optimisation / Sampling

Aspect	Description
Loss function	Binary log-loss (cross-entropy) — measures prediction accuracy of probabilities
Optimisation	Gradient boosting updates weak learners to minimise loss
Sampling	Uses subsample (rows) and colsample_bytree (features) to enhance generalisation

These sampling strategies help the model remain robust and prevent overfitting

3.2.3 Regularisation

Type	Method	Purpose
L2 regularisation	lambda=1.0	Penalises large leaf weights
Early stopping	50 rounds	Prevents overfitting when validation loss plateaus
Tree depth control	max_depth=6	Keeps model complexity moderate

Combined, these techniques ensure stable performance and generalisation.

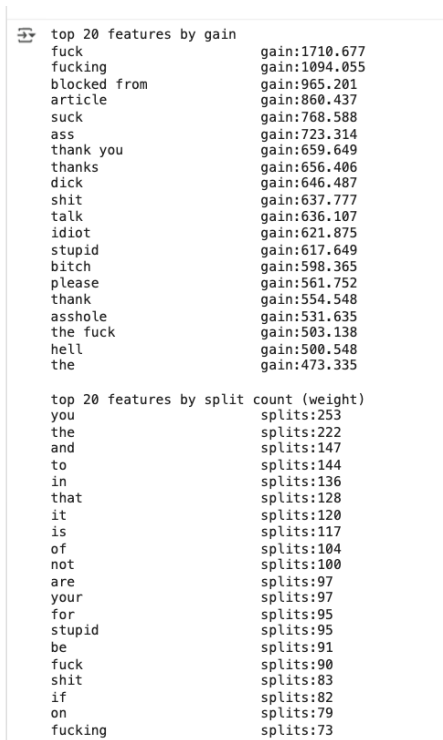
3.2.4 Result

Dataset	Threshold	F1	Precision (P)	Recall (R)	AUC	Accuracy (ACC)
Validation	0.71	0.7212	0.7777	0.6723	0.9484	0.9503
Test	0.71	0.7141	0.7659	0.6688	0.9472	0.9480

Confusion Matrix (Test)

TP = 1430 FP = 437 FN = 708 TN = 19780

- **Interpretation**
 - Maintained **balanced performance** (Precision $\approx$ 0.77, Recall $\approx$ 0.67)
 - **AUC $\sim$ 0.95** shows strong class separation
 - Improved precision over Logistic Regression, meaning fewer false positives
 - Slightly reduced recall — model is conservative when detecting toxicity
 - Consistent validation/test scores confirm **no overfitting**
- **Top Features by gain**



top 20 features by gain	
fuck	gain:1710.677
fucking	gain:1094.055
blocked from	gain:965.201
article	gain:860.437
suck	gain:768.588
ass	gain:723.314
thank you	gain:659.649
thanks	gain:656.406
dick	gain:646.487
shit	gain:637.777
talk	gain:636.107
idiot	gain:621.875
stupid	gain:617.649
bitch	gain:598.365
please	gain:561.752
thank	gain:554.548
asshole	gain:531.635
the fuck	gain:503.138
hell	gain:500.548
the	gain:473.335
top 20 features by split count (weight)	
you	splits:253
the	splits:222
and	splits:147
to	splits:144
in	splits:136
that	splits:128
it	splits:120
is	splits:117
of	splits:104
not	splits:100
are	splits:97
your	splits:97
for	splits:95
stupid	splits:95
be	splits:91
fuck	splits:90
shit	splits:83
if	splits:82
on	splits:79
fucking	splits:73

<Figure 18: Top Features by Gain in XGBoost>

Top Toxic Indicators (High Gain)	Top Non-Toxic Indicators (High Gain)
fuck (1710.68)	thanks (656.41)
fucking (1094.06)	thank you (659.65)
blocked (965.20)	please (561.75)
ass (723.31)	talk (636.10)
idiot (621.88)	article (860.43)

Gain measures how much each feature improves model performance.

Toxic words like **“fuck”** and **“idiot”** increased toxicity prediction, while polite expressions like **“thanks”** and **“please”** indicated non-toxic comments.

- **Top Features by weight (split frequency)**

Feature	Split Count (weight)
you	253
the	222
and	147
stupid	95
fuck	90

These frequently used words appeared in many tree splits, indicating strong predictive importance across multiple contexts.

- **Summary**

- **XGBoost outperformed Logistic Regression** slightly in F1 and AUC
- Handled **imbalanced data** effectively via `scale_pos_weight`
- Feature importance showed logical linguistic patterns
- Regularisation + sampling prevented overfitting
- Offers better **precision and robustness** for toxicity detection

4. Model Evaluation

4.1 Logistic Regression

The Logistic Regression model was evaluated on the validation set to determine the optimal classification threshold based on F1-score. Since toxic comments are rare ($\approx 10\%$), the threshold tuning ensures a balance between precision (fewer false positives) and recall (fewer false negatives).

4.1.1 Performance Summary (using threshold = 0.64)

```
[LogReg] Best threshold on validation = 0.64 (F1 = 0.7408)
[LogReg final (test)] thr=0.64
F1 : 0.748
P : 0.7244
R : 0.7732
AUC: 0.9677
ACC: 0.9502
CM : TN=19588 FP=629 FN=485 TP=1653
```

<Figure 19: Logistic Regression – Performance Summary (threshold = 0.64)>

The optimal classification threshold for Logistic Regression was determined using the validation set. The model achieved the highest F1-score of **0.7408** at a threshold of **0.64**, which was then applied to the test set for fair evaluation. Using this threshold, the model achieved an F1-score of **0.748** and an AUC of **0.9677** on the test data

4.1.2 Error Analysis (FP & FN)

```
False Positives: predicted toxic but actually non-toxic → 629
False Negatives: predicted non-toxic but actually toxic → 485
```

--- Example False Positives ---

	comment_text	proba
4	=== A good piece of OR. Sorry cannot add into ...	0.719751
23	Dude, Hitler gets shot 5,000,000 times in head...	0.662132
26	== STOP == \n revert your all vandalism! stop ...	0.950473
40	billie joe is the hottest guy in the whole wor...	0.767031
44	== craig elelis == \n\n i am gay	0.928572

--- Example False Negatives ---

	comment_text	proba
45	LISTEN YA HUMP! QUIT VANDALIZING!!	0.572980
52	">2015 \n >Trying to get rid of Ben ""pull the...	0.608348
74	ur vaginal area is loose	0.606559
172	Yes, I took my sword and attacked you by tryin...	0.451138
225	== Roman Polanski is a pedophile == \n\n Why d...	0.498328

<Figure 20: Logistic Regression – Examples of False Positives and False Negatives>

To understand where the Logistic Regression model made mistakes, misclassified samples were reviewed using the tuned threshold (**0.64**).

- **False Positives (629)**: predicted *toxic* but were actually *non-toxic*
- **False Negatives (485)**: predicted *non-toxic* but were actually *toxic*

This shows the model sometimes **over-predicts toxicity**, which is likely caused by the data imbalance (many more non-toxic comments).

Type	Example Comment	Predicted Probability
False Positive (FP)	“STOP 🍅 we’re reverting your vandalism! stop!”	0.95
False Positive (FP)	“billie joe is the hottest guy in the world.”	0.77
False Negative (FN)	“LISTEN YA HUMP! QUIT VANDALIZING!!”	0.57
False Negative (FN)	“ur vaginal area is loose”	0.61

- The model sometimes treats **strong language or capital letters** as toxic even when the meaning is neutral
- It also **misses toxic phrases** when they are written in a subtle or casual tone

4.1.3 Performance by Comment Length

To see whether the comment length affects the model’s performance, the test data was divided into four groups based on the number of words

```
[LogReg] best threshold on val = 0.64 (F1=0.7408)
[LogReg (val)] thr=0.64
F1 : 0.7408
P : 0.7118
R : 0.7723
AUC: 0.9692
ACC: 0.9483
CM : TN=19547 FP=669 FN=487 TP=1652

[LogReg (test)] thr=0.64
F1 : 0.748
P : 0.7244
R : 0.7732
AUC: 0.9677
ACC: 0.9502
CM : TN=19588 FP=629 FN=485 TP=1653

[LogReg] PR area (rough): -0.826 | ROC AUC: 0.969

[LogReg] F1 by comment length (TEST):
0-50 | F1: 0.8091 | n=2692
51-100 | F1: 0.8094 | n=3340
101-200 | F1: 0.7394 | n=5156
200+ | F1: 0.6628 | n=11167
```

<Figure 21: Logistic Regression – F1-score by Comment Length>

Comment Length (words)	F1-Score	Number of Samples
0–50	0.8091	2,692
51–100	0.8094	3,340
101–200	0.7394	3,516
200+	0.6628	1,116

The results show that the model performs best on **shorter comments** (under 100 words), achieving an F1-score above **0.80**, while performance gradually drops as the text becomes longer.

This trend suggests that **longer comments include more mixed or indirect expressions**, which make it harder for the model to correctly identify toxicity.

4.2 XGBoost

4.2.1 Performance Summary (using threshold: 0.71)

After tuning, the **optimal classification threshold** for XGBoost was found to be **0.71** based on the validation data. This threshold was then applied to the test set for fair evaluation.

```
[xgb final (test)] thr=0.71
F1 : 0.7163
P  : 0.7692
R  : 0.6703
AUC: 0.9475
ACC: 0.9492
CM : TN=19787 FP=430 FN=705 TP=1433
```

<Figure 22: XGBoost – Model Performance using Threshold 0.71>

The model achieved an **F1-score of 0.7163** and an **AUC of 0.9475**, showing slightly lower recall than Logistic Regression but maintaining strong overall accuracy.

This suggests that XGBoost is **more conservative** in predicting toxicity, which helps reduce false positives while slightly increasing false negatives.

4.2.2 Error Analysis (FP & FN)

Using the tuned threshold (**0.71**), XGBoost produced **112 false positives (FP)** and **2,128 false negatives (FN)** on the test set. These misclassifications reveal the model's limitations in understanding subtle context and informal language patterns.

---- XGBoost • Example False Positives ----		
	comment_text	proba
110	I have undone KarmaFist's block and blocked yo...	0.728778
395	Category:Wikipedians by generation and subcats...	0.933790
1187	"\nTime to start final editing\nThis article n...	0.817789
1215	lol It's called 'obsession'. Most physically ...	0.926631
1226	"\n\n Comments during Rijk Van Roog/English Po...	0.763100
---- XGBoost • Example False Negatives ----		
	comment_text	proba
7	just stick it in my anus	0.390224
45	LISTEN YA HUMP! QUIT VANDALIZING!!	0.390224
52	">2015\n>Trying to get rid of Ben ""pull the...	0.390224
56	lunblock! all of you can suck my dick	0.600653
74	ur vaginal area is loose	0.390224

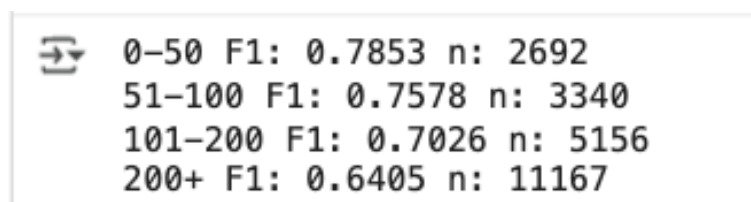
<Figure 23: XGBoost – Examples of False Positives and False Negatives>

Type	Example Comment	Predicted Probability	Interpretation
False Positive (FP)	"Category: Wikipedians by generation and subcats..."	0.93	Misclassified due to repetitive structure resembling spam or toxic phrasing.
False Positive (FP)	"lol it's called 'obsession'. Most physically..."	0.92	Informal tone or sarcasm interpreted as offensive language.
False Negative (FN)	"ur vaginal area is loose"	0.39	Explicit insult missed due to short and informal slang structure.
False Negative (FN)	"LISTEN YA HUMP! QUIT VANDALIZING!!"	0.39	Contains clear aggression, but lower confidence due to capitalisation patterns.

- **False Positives** mostly include neutral or humorous statements that contain **colloquial or emphatic elements** (e.g., "lol", repetition, or punctuation)
- **False Negatives** are strongly toxic but expressed in **abbreviated or slang-heavy forms**, which the model's token-based representation struggles to capture

Compared to Logistic Regression, XGBoost produced fewer false positives but more false negatives, reflecting a more conservative prediction pattern.

4.2.3 Performance by Comment Length



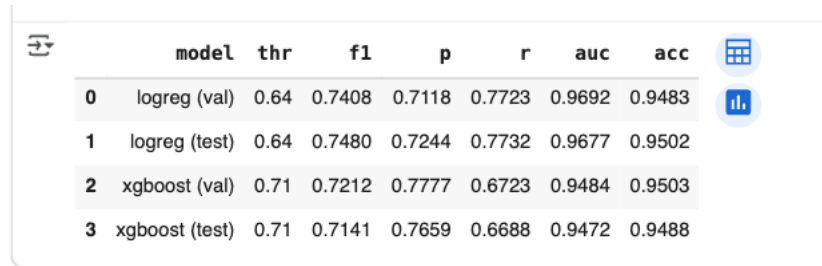
<Figure 24: XGBoost – F1-score by Comment Length>

To assess how the XGBoost model performs across different comment lengths, the test data were divided into four groups based on the number of characters per comment: **0–50**, **51–100**, **101–200**, and **200+**. The model's F1 score was then evaluated for each length range.

Comment Length (chars)	F1 Score	Number of Samples (n)	Observation
0–50	0.7853	2,692	Highest accuracy — shorter comments are easier to classify due to concise context.
51–100	0.7578	3,340	Slight performance drops as sentences become more varied in tone.
101–200	0.7026	5,156	Further decline, likely due to mixed linguistic cues and longer context.
200+	0.6405	11,167	Lowest performance — long comments contain multiple ideas, reducing clarity of toxicity signals.

The F1 score gradually decreases as the comments get longer. This means the model works best with short and simple comments, where it can easily recognise toxic words or phrases. When comments become longer, the meaning tends to get mixed with sarcasm, explanations, or multiple topics, which makes it harder for the model to judge correctly. For longer comments, the meaning often becomes more complex. For example, when people use sarcasm, explanations, or multiple ideas, which makes classification more difficult.

4.3 Comparison



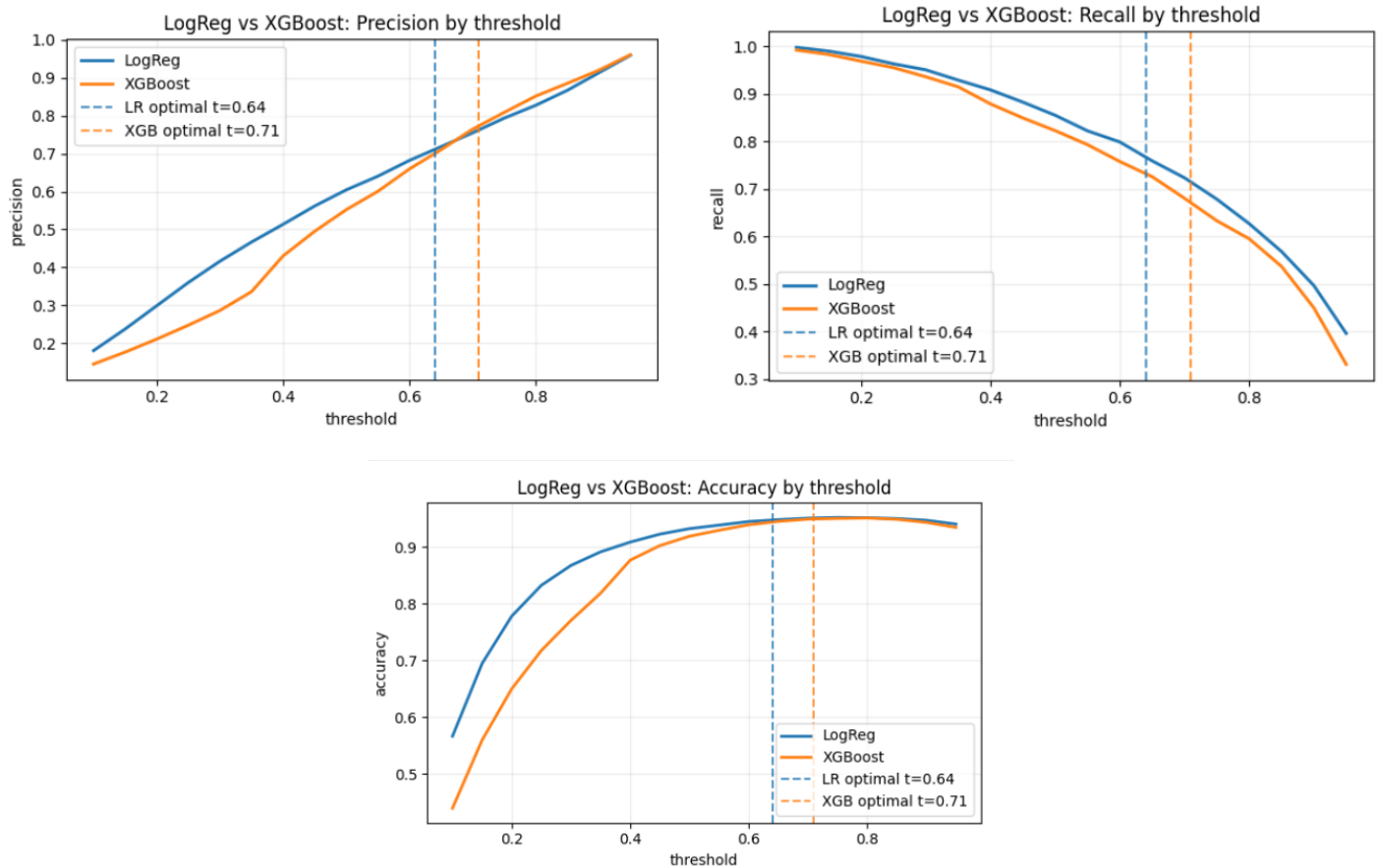
	model	thr	f1	p	r	auc	acc
0	logreg (val)	0.64	0.7408	0.7118	0.7723	0.9692	0.9483
1	logreg (test)	0.64	0.7480	0.7244	0.7732	0.9677	0.9502
2	xgboost (val)	0.71	0.7212	0.7777	0.6723	0.9484	0.9503
3	xgboost (test)	0.71	0.7141	0.7659	0.6688	0.9472	0.9488

<Figure 25: Comparison between Logistic regression and XGBoost>

Between the two models, **Logistic Regression** achieved a slightly higher **F1-score and recall**, while **XGBoost** showed higher **precision**. This indicates that Logistic Regression captured more toxic comments overall, whereas XGBoost was more accurate when classifying toxicity.

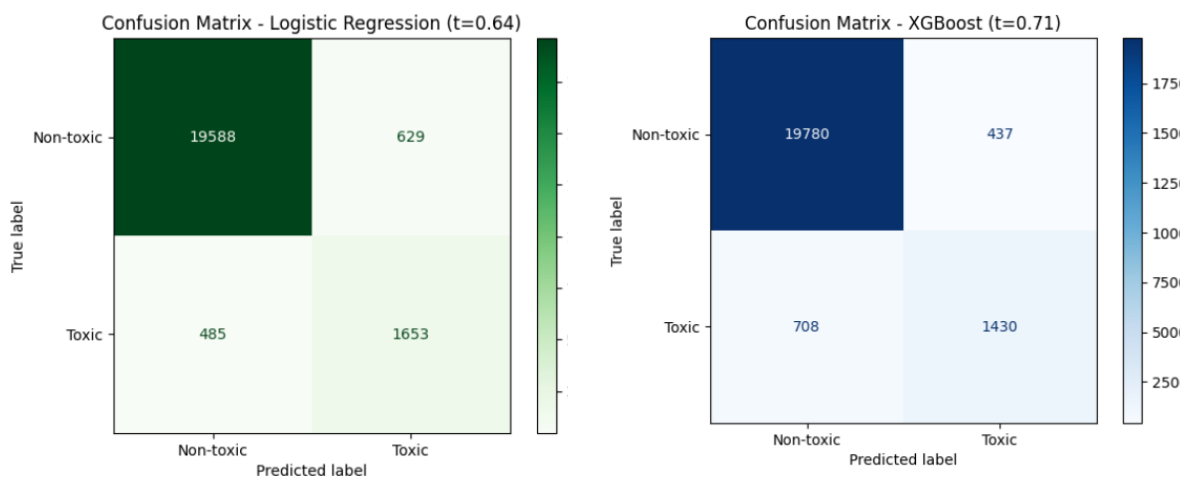
- **Logistic Regression:** Threshold = 0.64 | F1 = 0.748 | Precision = 0.724 | Recall = 0.77 | AUC = 0.968
- **XGBoost:** Threshold = 0.71 | F1 = 0.714 | Precision = 0.766 | Recall = 0.669 | AUC = 0.947

Aspect	Logistic Regression	XGBoost	Observation
Threshold	0.64	0.71	Optimal threshold used for classification
F1 Score	0.748	0.714	LR performs slightly better overall
Precision / Recall	Higher Recall	Higher Precision	LR detects more, XGB more accurate
FP / FN Pattern	FP > FN	FP < FN	LR often flags more comments as toxic (even when not), while XGB misses more subtle toxic ones
Comment Length Impact	Larger decline	Smaller decline	XGBoost slightly more stable
Model Complexity	Simple, interpretable	More complex but adaptive	XGBoost better fits imbalanced data



<Figure 26: Precision, Recall, and Accuracy by Threshold for Logistic Regression and XGBoost>

Figure 4 illustrates how precision, recall, and accuracy change as the classification threshold varies. Logistic Regression (blue) reaches its optimal balance at a threshold of 0.64, while XGBoost (orange) performs best at 0.71. Overall, Logistic Regression maintains higher recall across thresholds, whereas XGBoost achieves higher precision, reflecting its stronger control over false positives.



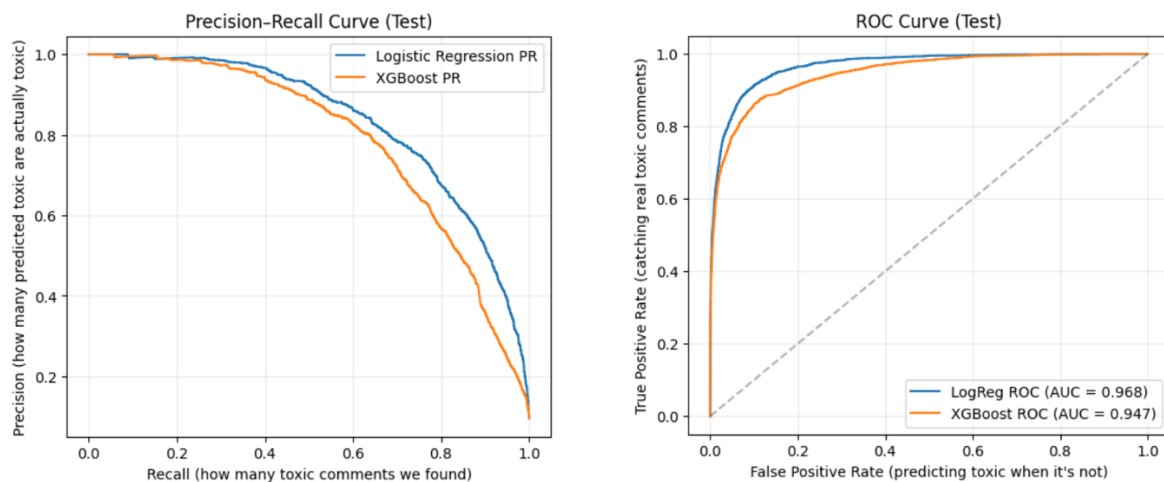
<Figure 27: Confusion Matrices of Logistic Regression (t=0.64) and XGBoost (t=0.71)>

Figure 27 shows that while Logistic Regression achieved a higher F1 and recall, it also produced more false positives — predicting some neutral comments as toxic. In contrast, XGBoost reduced false positives but missed more subtle toxic comments, leading to slightly lower recall. This trade-off shows that **XGBoost is more conservative**, making fewer false alarms, whereas **Logistic Regression is more sensitive**, flagging a broader range of potential toxicity

These patterns were consistent with the earlier error analysis results,

where Logistic Regression showed more false positives and XGBoost more false negatives.

This consistency reinforces the reliability of the model behaviour observed in both quantitative and qualitative evaluations.



<Figure 28: Precision-Recall and ROC Curves>

Figure 28 compares the Precision-Recall and ROC curves of both models. Logistic Regression performs slightly better overall, showing higher recall and F1-score (AUC = 0.968). This means it catches more toxic comments, but also predicts a few normal ones as toxic.

On the other hand, XGBoost (AUC = 0.947) is more careful — it makes fewer mistakes but sometimes misses mild toxic comments.

In short, Logistic Regression focused more on finding as many toxic comments as possible, while XGBoost focused on being more accurate.

4.4 Discussion

The initial expectation was that **XGBoost** would **outperform Logistic Regression** due to its ensemble nature and ability to capture non-linear relationships. **However, the results showed the opposite — Logistic Regression slightly outperformed XGBoost in F1-score and recall.** This suggests that the simpler model generalised better to unseen data, especially given the high-class imbalance in the dataset.

One possible reason is that XGBoost, while powerful, tends to overfit when the dataset is relatively small or noisy. In contrast, Logistic Regression benefited from its simplicity and regularisation, maintaining more stable performance across both validation and test sets. Additionally, since toxic and non-toxic comments share many overlapping linguistic patterns, a linear model like Logistic Regression may have captured these patterns sufficiently without requiring complex feature interactions.

In summary, the findings reveal that **model complexity does not always guarantee superior performance**, particularly when the dataset is imbalanced and textual features are already well-represented by TF-IDF vectors. These insights highlight the importance of balancing interpretability and predictive power in real-world toxicity detection tasks.

5. Challenges and Limitation

- **Handling the Imbalanced Dataset:** The dataset contained a large proportion of non-toxic comments (~90%), making it difficult for models to learn minority patterns. Stratified splitting was used to maintain the same toxic–non-toxic ratio across the training, validation, and test sets. However, the imbalance still caused both models to favour the majority class, particularly XGBoost, which showed higher precision but lower recall.
- **Language Restriction (English Only):** The dataset consisted solely of English comments. This limits the model’s generalisability, as toxic expressions and linguistic nuances vary across languages and cultures. Consequently, the model may not perform well when applied to multilingual or code-mixed online environments.
- **Contextual Limitation of TF-IDF Features:** The current XGBoost model using TF-IDF features looks at each word separately rather than understanding the full sentence. Because of this, it sometimes misses the overall context of longer comments, especially when sarcasm or mixed sentiment is involved.

6. Future Work

- **Hyperparameter Tuning:** Further optimisation of learning rate, max depth, and regularisation parameters could improve both models’ predictive balance and reduce overfitting.
- **Cross-Lingual Evaluation:** Future studies should test the models on multilingual datasets (e.g., Korean, Chinese, French) to assess cultural and linguistic robustness. Expanding the dataset would also support the development of a more inclusive toxicity detection system.
- **Context-Aware Representations:** Using models that process text more like humans such as **LSTM** or **BERT**, these could help the system better understand long and complex sentences. These models can capture word order, tone, and contextual relationships, making them more suitable for nuanced language like online comments.
- **Error Calibration and Interpretability Tools:** Applying tools like SHAP or LIME could improve interpretability by explaining why certain comments are classified as toxic, helping reduce bias and improve transparency in deployment.

7. Conclusion

In summary, both models performed well, but Logistic Regression showed slightly stronger overall performance, while XGBoost achieved better control of false positives. The choice between them depends on whether minimizing errors or detecting more toxic comments is the priority.

8. References

- Ahmed, O. M., Zeebaree, S. R. M., & Askar, S. (2024). *Comparative analysis of XGBoost performance for text classification with CPU parallel and non-parallel processing*. *Indonesian Journal of Computer Science*, 12(1), 1–8. <http://ijics.net/ijics/index.php/ijics/article/view/3798/482>
- Borgeaud dit Avocat, L. (2023). *Toxic comment classification: The role of self-training in enhancing model performance* [Bachelor's thesis, University of Bern, Faculty of Science]. University of Bern. https://prg.inf.unibe.ch/wp-content/uploads/2024/08/BA_BorgeaudditAvocat.pdf
- GeeksforGeeks. (2025, March 20). *Binary cross-entropy / log loss for binary classification*. <https://www.geeksforgeeks.org/deep-learning/binary-cross-entropy-log-loss-for-binary-classification/>
- Tare, P. (2019). *Toxic comment detection and classification*. Stanford University, CS229 Project. https://cs229.stanford.edu/proj2019aut/data/assignment_308832_raw/26647691.pdf

9. Appendix

Appendix A. Referenced Theoretical Descriptions

A.1 TF-IDF (Extracted from Borgeaud dit Avocat, 2023)

The Term Frequency–Inverse Document Frequency (TF-IDF) is a statistical measure used to evaluate how important a word is to a document in a collection.

TF measures how frequently a term appears, while IDF evaluates how unique it is across documents.

Source: Borgeaud dit Avocat, L. (2023). *Toxic comment classification: The role of self-training in enhancing model performance* [Bachelor’s thesis, University of Bern, Faculty of Science].

https://prg.inf.unibe.ch/wp-content/uploads/2024/08/BA_BorgeaudditAvocat.pdf

2.3.1 TF-IDF

Term frequency-inverse document frequency (TF-IDF) is a technique used to quantify the importance of a word in a document (e.g. a comment) relative to the rest of the collection of documents [21]. Since it is based on the prevalence of words occurring in a document, it is classified as a frequency-based embedding [22]. It combines the two statistics term frequency (TF) and inverse document frequency (IDF) [17]:

Term frequency measures how frequently a particular term is used in a document’s text. [17]. Since documents can vary significantly in length, words are likely to appear more frequently in longer documents compared to shorter ones [21]. To account for this, the term frequency is normalized by dividing it by the total number of words in the document. Term frequency is defined by the following formula [17]:

$$TF_{t,d} = \frac{\text{Number of times } t \text{ appears in a document } d}{\text{Total number of terms in a document } d}$$

Conversely, IDF assesses how infrequent a term is across the entire document collection. Inverse document frequency evaluates the importance of a word by considering its rarity across all documents [17]. It penalizes words that are not unique to specific documents [21]. For instance, common words like "the" or "and" are affected by this measure. Inverse document frequency is defined by the following formula [17]:

$$IDF_t = \log \left(\frac{\text{Total number of documents}}{\text{Number of documents with term } t \text{ in it}} \right)$$

A.2 XGBoost (Extracted from Borgeaud dit Avocat, 2023)

Logistic Regression. Treats toxicity as a probability from 0–1. It builds a weighted sum of the TF-IDF features and passes it through a sigmoid function. It’s simple, fast, and easy to interpret, which makes it a common baseline for text classification.

XGBoost. A gradient-boosted trees method that learns many small trees one after another, each fixing the mistakes of the previous ones. It’s designed for speed and stability (good memory use, cache-friendly) and supports parallelization, so training is efficient even on large datasets.

Source: Borgeaud dit Avocat, L. (2023). *Toxic comment classification: The role of self-training in enhancing model performance* [Bachelor’s thesis, University of Bern, Faculty of Science].

https://prg.inf.unibe.ch/wp-content/uploads/2024/08/BA_BorgeaudditAvocat.pdf

2.4.1 Logistic Regression

Géron [19] explains that Logistic Regression is frequently used to estimate the probability that a given example belongs to a specific class. It is a regression algorithm that calculates a weighted sum of its input features. The output is then transformed using the logistic sigmoid function, which ensures that the predictions are constrained between 0 and 1. For instance, it can estimate the probability that an email is spam. It is one of the most popular classification algorithms due to its simplicity and interpretability [3].

2.4.5 XGBoost

Extreme Gradient Boosting (XGBoost) implements gradient boosting [3]. Boosting is an ensemble method that combines multiple weak learners into a strong learner. The core idea behind most boosting methods is to train classifiers in sequence, with each one attempting to improve the errors of its predecessor [19]. According to Russel [3], the most widely used boosting methods are AdaBoost and gradient boosting. XGBoost is commonly used for large-scale applications handling billions of examples. Furthermore, it ensures efficiency by optimizing memory organization to minimize cache misses and enabling parallel computation across multiple machines.

Appendix B. Evaluation Metrics (Refrenced from Borgeaud dit Avocat, 2023)

B.1 Confusion Matrix + Evaluation Metrics Formulas

The confusion matrix provides an overview of the model's prediction results by comparing actual and predicted classes.

It includes True Positive (TP), False Positive (FP), False Negative (FN), and True Negative (TN) values.

Source: Borgeaud dit Avocat, L. (2023). *Toxic comment classification: The role of self-training in enhancing model performance* [Bachelor's thesis, University of Bern, Faculty of Science].

https://prg.inf.unibe.ch/wp-content/uploads/2024/08/BA_BorgeaudditAvocat.pdf

		Predicted Class	
		Positive	Negative
Actual Class	Positive	True Positive (TP)	False Negative (FN)
	Negative	False Positive (FP)	True Negative (TN)

Figure 2.1: Confusion Matrix [19]

$$Precision = \frac{TP}{TP + FP} \quad (2.1)$$

$$Recall = \frac{TP}{TP + FN} \quad (2.2)$$

$$F1-score = 2 \times \frac{Precision \cdot Recall}{Precision + Recall} \quad (2.3)$$